Refinamento de Testes com GenAI em Testes de Software

Projeto	Projeto	Autor
ServeRest API	ServeRest	Arlen Freitas

📌 1. Introdução
🎯 2. Objetivo
💼 3. Ferramentas Utilizadas
📋 4. Contexto do Plano de Testes Original
- Escopo coberto
- Métricas
- Tipos de testes

🔍 5. Análise Crítica do Plano Original
- Pontos Fortes
- ⚠️ Gaps Identificados
  - Arquitetura
  - Segurança
  - Contrato
  - Resiliência
  - Dados
  - Cobertura

🤖 6. Uso de GenAI e Prompt Engineering
- 🧠 6.1 Estratégia de Prompt
- 6.2 Prompts Utilizados
  - Prompt 1 — Refatoração Arquitetural
  - Prompt 2 — Expansão de Cobertura de Testes
  - Prompt 3 — Contract Testing e Qualidade
  - Prompt 4 — Resiliência, Logs e Performance
  - Prompt 5 — Organização, Tags e CI/CD
- 📊 6.3 Resultado do Uso de Prompt Engineering
- 🧠 6.4 Aprendizado com Prompt Engineering

🏗️ 7. Arquitetura Evoluída com GenAI
- Evoluções importantes

🚀 8. Melhorias Propostas GenAI
- Arquitetura
- Segurança
- Contrato
- Resiliência
- Observabilidade
- Automação
- Performance

✅ 9. Checklist de Refinamento

Autenticação
Usuários
Produtos
Carrinho
Contrato
Resiliência
Observabilidade
Performance
📊 10. Comparativo Antes vs Depois
📈 11. Ganhos Obtidos
🧾 12. Conclusão
🧠 13. Lições Aprendidas

📌 1. Introdução

Este documento apresenta o refinamento do plano de testes da API ServeRest utilizando técnicas de Inteligência Artificial Generativa (**GenAI**) **(Amazon Q)** e **Prompt Engineering**.

A proposta foi evoluir um plano de testes previamente desenvolvido, aplicando boas práticas de engenharia de qualidade, automação e arquitetura de testes.

🎯 2. Objetivo

- Refinar o plano de testes existente
- Aplicar Prompt Engineering com GenAI
- Melhorar cobertura, qualidade e robustez
- Identificar gaps técnicos
- Propor melhorias práticas e escaláveis

🧰 3. Ferramentas Utilizadas

- Amazon Q (**GenAI**)
- Robot Framework
- RequestsLibrary
- Collections / String Libraries

📋 4. Contexto do Plano de Testes Original

O plano original contemplava:

Escopo coberto

- Autenticação (Login)
- Cadastro de usuários
- Busca de usuários
- Atualização de usuários

Métricas

- Total de cenários: 12
- Executados: 100%
- Sucesso: 10
- Falhas: 2
- Bugs encontrados: 1 (BUG-12)

Tipos de testes

- Testes positivos
- Testes negativos
- Validação de contrato básica

🔍 5. Análise Crítica do Plano Original

Pontos Fortes

- Boa cobertura inicial de usuários
- Uso de automação com Robot Framework
- Identificação de bug crítico (validação de campos)
- Organização básica de testes

⚠️ Gaps Identificados

Arquitetura

- Ausência de camada de serviços
- Baixa separação de responsabilidades

Segurança

- Não havia testes de autenticação avançada
- Ausência de testes de autorização

Contrato

- Validação superficial (sem schema estruturado)

Resiliência

- Sem retry
- Sem tratamento de falhas intermitentes

Dados

- Dados fixos (não dinâmicos)

Cobertura

- Produtos e carrinho fora do escopo
- Sem testes de performance

🤖 6. Uso de GenAI e Prompt Engineering

Para o refinamento do plano de testes, foi utilizado o **Amazon Q** com uma abordagem iterativa de Prompt Engineering.

A estratégia adotada consistiu em:

- Definição clara de contexto
- Atribuição de papéis especializados (**QA, SDET, Arquiteto**)
- Execução em múltiplas etapas (refinamento progressivo)
- Foco em geração de código e arquitetura real

🧠 6.1 Estratégia de Prompt

Os prompts foram aplicados de forma incremental, evoluindo o nível de maturidade do projeto:

1. Estruturação e arquitetura
2. Implementação de melhorias
3. Expansão de cobertura (segurança, edge cases, performance)
4. Refinamento final

6.2 Prompts Utilizados

Prompt 1 — Refatoração Arquitetural

```
Act as a Senior QA Automation Engineer, Test Architect, and SDET with d

Context:
We have an existing Robot Framework project testing the ServeRest API w

Challenge_03/
├── resources/
├── tests/

The project already uses:
- RequestsLibrary
- Gherkin-style (Given/When/Then)

Goal:
Refactor and enhance this project to a production-grade, scalable, and

Tasks:
1. Refactor project structure separating:
   - services/
   - schemas/
   - data/
   - utils/

2. Create service layer:
   - Move all HTTP calls to service files
   - Create reusable keywords (Create User, Login User, etc.)

3. Improve session management:
   - Create Authenticated Session
   - Handle token reuse

4. Improve test structure:
   - Use Given/When/Then consistently
   - Remove duplication
   - Add Setup/Teardown

Constraints:
- Use Robot Framework
- Use RequestsLibrary
- Keep modular architecture

Output:
- Folder structure
- Refactored files
- Code examples
```

Prompt 2 — Expansão de Cobertura de Testes

```
Continue improving the framework.

Tasks:
1. Add advanced test scenarios:
   - Positive
   - Negative
   - Edge cases

2. Add security tests:
   - Invalid token
```

```
   - Missing token
   - Unauthorized access

3. Add validation scenarios:
   - Invalid data formats
   - Boundary values

4. Improve data handling:
   - Use external JSON files
   - Avoid hardcoded data

5. Generate complete Robot Framework test cases

Output:
- New test suites
- Example test cases
- Data-driven tests
```

Prompt 3 — Contract Testing e Qualidade

```
Enhance the API contract testing.

Tasks:
1. Create JSON Schema files for:
   - Login
   - Users
   - Products
   - Cart

2. Implement validation keyword:
   Validate Response Against Schema

3. Validate:
   - Required fields
   - Data types
   - Unexpected fields

4. Improve error validation:
   - Standard error response structure

Output:
- JSON schema examples
- Validation keywords
- Contract test examples
```

Prompt 4 — Resiliência, Logs e Performance

```
Improve non-functional aspects of the framework.

Tasks:
1. Implement retry mechanism:
   - Retry Request Until Success

2. Improve logging:
   - Log request payload
   - Log response body
   - Log status codes

3. Add performance checks:
   - Validate response time
   - Add simple load scenarios

```

```
4. Improve error handling:
   - Detailed error messages
   - Debug-friendly logs

Output:
- Utility keywords
- Logging improvements
- Performance test examples
```

Prompt 5 — Organização, Tags e CI/CD

```
Finalize the framework for enterprise readiness.

Tasks:
1. Add tagging strategy:
   - smoke
   - regression
   - security
   - performance

2. Organize test suites:
   - Separate by domain and purpose

3. Suggest CI/CD pipeline:
   - Smoke on push
   - Regression on PR
   - Performance scheduled

4. Optimize execution:
   - Parallel execution strategy
   - Test grouping

Output:
- Tagging strategy
- CI/CD structure
- Execution commands
```

📊 6.3 Resultado do Uso de Prompt Engineering

A aplicação iterativa dos prompts resultou em:

- Arquitetura escalável baseada em camadas
- Aumento significativo da cobertura de testes
- Inclusão de testes de segurança, performance e edge cases
- Implementação de validação de contrato robusta
- Melhoria na Observabilidade e resiliência

🧠 6.4 Aprendizado com Prompt Engineering

- Prompts específicos geram resultados mais relevantes
- Iteração é essencial para maturidade do output

- Definir contexto + papel melhora drasticamente a qualidade
- Solicitar implementação (não sugestão) gera outputs mais práticos

---

🏗️ 7. Arquitetura Evoluída com GenAI

Foi proposto pela GenAI uma arquitetura escalável:

```
services/   → chamadas HTTP
resources/  → camada Gherkin
schemas/    → contratos JSON
data/       → dados externos
utils/      → utilidades (retry, log, contract)
tests/      → cenários organizados
```

Evoluções importantes

- Separação total de responsabilidades
- Camada de serviços desacoplada
- Validação de contrato robusta
- Testes orientados a dados
- CI/CD integrado

🚀 8. Melhorias Propostas GenAI

Arquitetura

- Implementação de service layer
- Modularização completa

Segurança

- Testes com token inválido
- Testes sem autenticação
- Validação de autorização

Contrato

- JSON Schema validation
- Validação de estrutura completa

Resiliência

- Retry automático
- Tratamento de timeout

Observabilidade

- Implementação de logs detalhados
- Request/Response tracking

Automação

- Data-driven testing
- Reutilização de keywords
- Setup/Teardown

Performance

- Validação de tempo de resposta
- Testes básicos de carga

✅ 9. Checklist de Refinamento

Autenticação

- Login com sucesso
- Login inválido
- Campos obrigatórios
- Token inválido
- Token expirado

Usuários

- Criar usuário válido
- Criar duplicado
- Validar email inválido
- Buscar inexistente
- Atualizar inexistente
- Deletar inexistente

Produtos

- Criar produto
- Criar sem auth
- Produto duplicado
- Preço inválido
- Buscar inexistente

Carrinho

- Adicionar produto
- Produto inexistente
- Checkout vazio
- Checkout sem auth
- Estoque insuficiente

Contrato

- Validar schema JSON
- Validar tipos
- Campos obrigatórios
- Campos extras
- Estrutura de erro

Resiliência

- Retry automático
- Timeout
- API indisponível

Observabilidade

- Implementação de logs detalhados
- Payloads dinâmicos
- Erros detalhados

Performance

- Tempo de resposta
- Carga básica
- Concorrência

📊 10. Comparativo Antes vs Depois

Aspecto	Antes	Depois
Cobertura	Parcial	Completa
Arquitetura	Simples	Escalável
Segurança	Não	Sim
Contrato	Básico	Schema completo
Resiliência	Não	Retry + Timeout
Performance	Não	Sim

📈 11. Ganhos Obtidos

- Aumento da cobertura de testes

- Redução de riscos em produção
- Melhor organização e manutenção
- Arquitetura escalável
- Testes mais confiáveis

🧾 12. Conclusão

A utilização de **GenAI** com técnicas de **Prompt Engineering** permitiu uma evolução significativa no plano de testes, tornando-o mais robusto, completo e alinhado com práticas modernas de **QA**.

A **IA** atuou como aceleradora, mas o refinamento técnico foi essencial para garantir qualidade e aplicabilidade real.

⚠️ **Importante!!!** O uso estratégico da **IA** não substitui o conhecimento técnico, mas potencializa a capacidade de análise e melhoria contínua dos testes.

As melhorias propostas neste documento representam um refinamento do plano de testes original e podem ser implementadas em etapas futuras para evolução contínua da automação.

Como validação prática das melhorias propostas, foi criada uma branch **(refactor/gen-ai)** separada contendo a implementação gerada com apoio de GenAI, demonstrando a viabilidade técnica das sugestões apresentadas neste documento.

🧠 13. Lições Aprendidas

- A qualidade do resultado depende do prompt
- IA potencializa, mas não substitui conhecimento técnico
- Refinamento iterativo gera melhores resultados
- Arquitetura de testes é tão importante quanto os testes